NDEJJE UNIVERSITY

Fear of God brings Knowledge and Wisdom

COURSE: COMPUTER SCIENCE

SEMESTER: II

COURSE UNIT: Cryptography and Cyber Security

FACILITATOR: MR.MUCHAKE BRIAN

NAME:KIRABIRA KEITH 23/2/306/D/387

Software Development Life Cycle (SDLC) Models & Quality Practices

1. Waterfall Model

The Waterfall Model is one of the earliest and simplest Software Development Life Cycle models. In this model, the development process flows sequentially from one phase to another, just like water flowing down a waterfall. Each phase must be completed before moving to the next phase.

Phases of the Waterfall Model

- Requirements Analysis
  In this stage, system requirements are gathered from users and stakeholders. These requirements are documented in a Software Requirement Specification (SRS).
- System Design
  The system architecture and design are created based on the requirements. Developers decide how the system will work internally.
- Implementation (Coding)
  Programmers write the actual code according to the design specifications.
- Testing
  The developed software is tested to detect errors, bugs, and defects.
- Deployment
  The system is delivered to the customer and installed in the real environment.
- Maintenance
  After deployment, the system may require updates, bug fixes, and improvements.

Advantages of the Waterfall Model

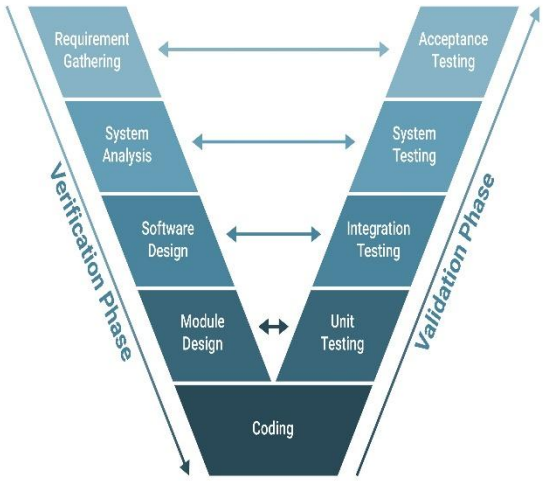
- Simple and easy to understand.
- Clear structure and documentation.
- Suitable for small projects with stable requirements.

Disadvantages

- Difficult to make changes after development begins.
- Testing happens late in the process.
- Not suitable for complex or dynamic projects.

2. V-Model (Verification and Validation Model)

The V-Model is an extension of the Waterfall Model where testing is planned in parallel with development. The model forms a V-shaped structure, where the left side represents development phases and the right side represents testing phases.



Phases of the V-Model

Left Side (Development Phases)

1. Requirement Analysis
2. System Design
3. Architecture Design
4. Module Design
5. Coding

Right Side (Testing Phases)

1. Unit Testing – tests individual components.
2. Integration Testing – tests combined modules.
3. System Testing – tests the entire system.
4. Acceptance Testing – ensures the system meets user requirements.

Advantages

- Testing is planned early.
- Errors can be detected earlier than in the Waterfall model.
- Provides better quality assurance.

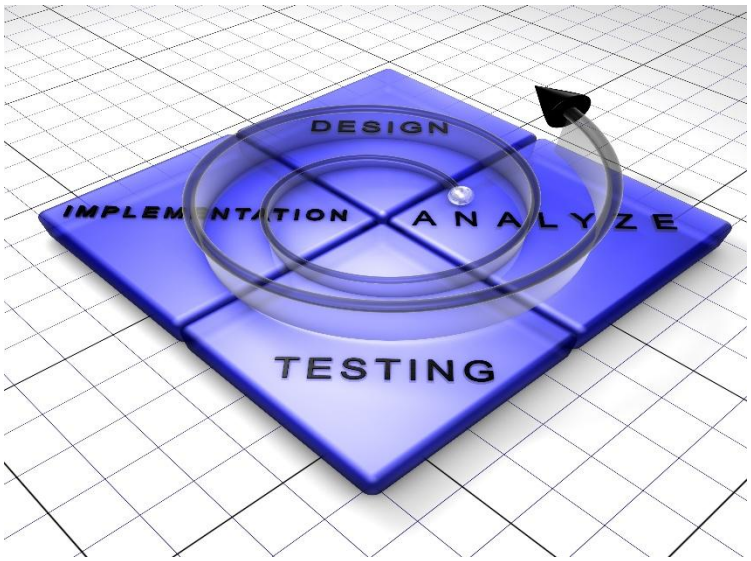
Disadvantages

- Still rigid like the Waterfall model.
- Not flexible for requirement changes.

3. Spiral Model

The Spiral Model combines elements of both Waterfall and iterative development models. It focuses heavily on risk analysis and is commonly used for large and complex projects.

The development process is represented as a spiral, where each loop represents a phase of the project.



Four Main Phases of the Spiral Model

- Planning
  Determine project objectives, alternatives, and constraints.
- Risk Analysis
  Identify potential risks and develop strategies to reduce them.
- Engineering
  Develop and test the software components.
- Evaluation
  The customer evaluates the progress and provides feedback.

This cycle continues until the project is completed.

Advantages

- Excellent for managing high-risk projects.
- Allows continuous user feedback.
- Flexible and adaptable.

Disadvantages

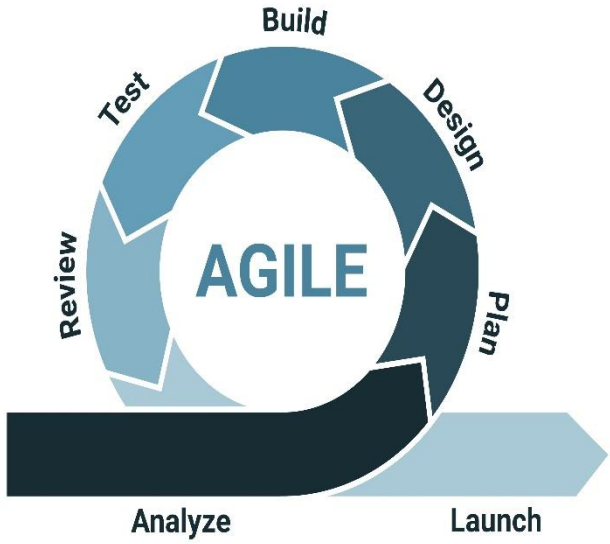
- Expensive and complex to manage.
- Requires strong risk analysis expertise.

---

4. Agile Methodology

Agile methodology is a modern software development approach that focuses on flexibility, collaboration, and rapid delivery of software. Instead of developing the entire system at once, Agile divides the project into small units called iterations or sprints.

Each sprint usually lasts 1–4 weeks and produces a working version of the software.



Key Principles of Agile

- Customer collaboration over contract negotiation.
- Working software over extensive documentation.
- Responding to change over following a fixed plan.
- Continuous improvement.

Common Agile Frameworks

- Scrum
- Kanban
- Extreme Programming (XP)

Agile Process

1. Planning
2. Sprint development
3. Testing
4. Review
5. Deployment
6. Feedback

Advantages

- Flexible and adaptable to change.
- Faster delivery of working software.
- High customer involvement.

Disadvantages

- Requires experienced teams.
- Difficult to estimate cost and time in large projects.

5. DevOps Quality Practices

DevOps is a modern approach that integrates software development (Dev) and IT operations (Ops) to improve collaboration and deliver software faster and with higher quality.

DevOps focuses on automation, continuous monitoring, and collaboration throughout the software development lifecycle.

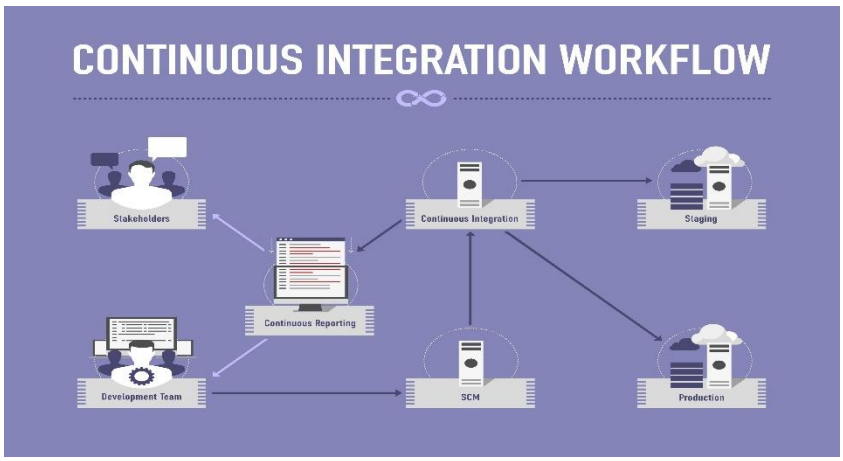
Key DevOps Quality Practices

- Automation
  Automated testing, building, and deployment help reduce human errors.
- Continuous Monitoring
  Systems are monitored in real time to detect issues quickly.
- Infrastructure as Code (IaC)
  Infrastructure is managed using code and automation tools.
- Collaboration Between Teams
  Developers and operations teams work together to improve productivity.
- Automated Testing
  Testing is performed automatically whenever code changes are made.

Benefits of DevOps

- Faster delivery of software.
- Improved reliability and stability.
- Better collaboration between teams.

6. Quality in Continuous Integration / Continuous Delivery (CI/CD)



Continuous Integration (CI) and Continuous Delivery (CD) are practices used to improve software quality by automatically integrating, testing, and deploying code changes.

Continuous Integration (CI)

Continuous Integration is a process where developers frequently merge their code changes into a shared repository. Every integration triggers automated builds and tests.

Benefits

- Early detection of bugs.
- Improved collaboration among developers.
- Faster development process.

Continuous Delivery (CD)

Continuous Delivery ensures that software can be released to production at any time. All code changes pass through automated testing and deployment pipelines.

Benefits

- Faster software releases.
- Reduced deployment risks.
- Higher product quality.

CI/CD Pipeline

A CI/CD pipeline typically includes:

- Code Commit
- Build Automation
- Automated Testing
- Integration Testing
- Deployment to Staging
- Deployment to Production

Conclusion

Software Development Life Cycle models such as Waterfall, V-Model, Spiral, and Agile provide structured approaches for building software systems. Modern development practices like DevOps and CI/CD focus on improving software quality, automation, and faster delivery. Choosing the right model depends on the project size, complexity, and requirements stability.